🏎️ TiDPI 07 — Rzetelny benchmark C++ i Python

📓 Tematy do dziennika

```
[TiDPI] Benchmark jako kontrolowany eksperyment – wspólne dane, zakres pomiaru i walidacja wyniku.
[TiDPI] Rzetelne pomiary w C++ i Pythonie – Release, rozgrzewka, powtórzenia i surowe wyniki.
[TiDPI] Porównanie implementacji C++ i Python oraz przygotowanie udokumentowanego raportu benchmarkowego.
```

🎯 Cel lekcji

Po tej lekcji będziecie potrafili zaprojektować i wykonać benchmark dwóch implementacji tak, aby porównanie miało określony sens, dawało się powtórzyć i nie nagradzało programu, który szybko obliczył niepoprawny wynik.

To jest finał pierwszego etapu kursu. Od prostego `start-stop` przeszliśmy przez deterministyczne dane, automatyczne uruchamianie, ustrukturyzowany output i walidację. Dzisiaj składamy te elementy w jeden eksperyment, który można pokazać bez rozpoczynania prezentacji od zdania: „Proszę nie patrzeć na trzeci pomiar, bo komputer chyba miał gorszy dzień”.

🔥 Dwa programy wchodzą na tor

Wyobraźcie sobie wyścig dwóch samochodów. Jeden jedzie po suchej autostradzie, drugi po błocie. Pierwszy wiezie kierowcę, drugi fortepian. Jeden startuje z rozpędu, drugi musi najpierw otworzyć garaż. Na mecie ogłaszamy: „Niebieski samochód jest szybszy!”.

A tak naprawdę to nie przeprowadziliśmy wyścigu, tylko urządziliśmy widowisko.

Z programami jest tak samo. Aby porównać implementację w C++ z implementacją w Pythonie, musimy ustalić co najmniej:

- wspólne zadanie i znaczenie wyniku;
- identyczne dane wejściowe;
- granice mierzonej operacji;
- konfigurację obu środowisk;
- liczbę rozgrzewek i właściwych prób;
- sposób sprawdzania poprawności;
- format zapisu surowych wyników;
- zasady formułowania wniosków.

> **Bon mot na dziś:** jeżeli zmierzyliśmy wszystko naraz, to najczęściej nie wiemy, co właściwie zmierzyliśmy. To benchmark w stylu „samochód, kierowca, tankowanie, korek i zakup hot doga — łącznie 47 minut”.

🧭 Benchmark w znaczeniu przyjętym w naszym kursie

W lekcji 00c rozróżniliśmy porównawczy test wydajnościowy od benchmarku. Test wydajnościowy może porównywać dwa stany aplikacji w tych samych, stabilnych warunkach: przed zmianą i po zmianie. Benchmark ma pokazać, ile dana implementacja potrafi osiągnąć w określonych warunkach — mówiąc mniej uroczyście: **ile fabryka dała**.

„Pełne obciążenie” nie oznacza jednak chaosu. Nadal potrzebujemy kontrolowanych warunków. Gaz wciskamy do podłogi, ale nie odkręcamy w tym samym czasie koła.

W tej lekcji benchmarkujemy pętlę obliczeniową wykonującą wielokrotnie tę samą operację na wspólnych danych. Nie testujemy dysku, czasu uruchamiania edytora, szybkości pisania poleceń ani refleksu osoby naciskającej Enter.

⚖️ Nie mierzymy „języka” w próżni

Zdanie „C++ jest 18,7 raza szybszy od Pythona” wygląda efektownie, ale bez dalszego opisu jest prawie bezwartościowe. Nie "uruchamiamy języków", tylko uruchamiamy konkretne programy:

- napisane przy użyciu określonych konstrukcji i algorytmów;
- wykonane przez konkretny kompilator albo interpreter;
- w określonej konfiguracji;
- na konkretnym systemie i sprzęcie;
- dla konkretnego zestawu danych.

Jeżeli w Pythonie użyjemy `sum()`, a w drugim wariancie ręcznej pętli, możemy w rzeczywistości porównywać kod wykonujący dużą część pracy wewnątrz szybkiej implementacji bibliotecznej z kodem interpretującym każdą iterację. Jeżeli w C++ użyjemy `std::sort`, a w Pythonie własnego sortowania bąbelkowego, porównamy przede wszystkim różne algorytmy. Wynik może być prawdziwy, lecz podpis pod nim będzie fałszywy.

Dlatego uczciwy wniosek brzmi na przykład:

> Dla wspólnego pliku danych, ręcznej pętli sumującej kwadraty, podanej liczby powtórzeń, kompilacji C++ w Release i wskazanej wersji Pythona implementacja C++ osiągnęła w tym środowisku niższą medianę czasu.

To zdanie jest dłuższe od „C++ miażdży Pythona”, ale za to nadaje się do raportu.

🧱 Anatomia rzetelnego benchmarku

1️⃣ Jednoznaczne zadanie

Obie implementacje mają obliczyć to samo. W demonstracji będzie to suma kwadratów wszystkich wartości, powtórzona określoną liczbę razy. Wynikiem jest `checksum`, czyli liczba kontrolna pozwalająca sprawdzić, czy program rzeczywiście wykonał pracę.

2️⃣ Wspólne dane wejściowe

Ten sam seed w C++ i Pythonie nie gwarantuje tego samego ciągu. Generatory i sposób przekształcania ich wyników mogą być inne. Dlatego dane generujemy raz i zapisujemy w pliku tekstowym, a oba programy odczytują ten sam plik.

Wspólny plik jest jak identyczny arkusz egzaminacyjny. Dwa arkusze z tym samym numerem na okładce, ale innymi pytaniami, nie tworzą uczciwego porównania.

3️⃣ Jawna granica pomiaru

Wczytanie danych odbywa się przed uruchomieniem zegara. Badamy pętlę obliczeniową, więc czas pracy dysku nie może przypadkiem wejść do wyniku.

```
// Mierzymy wyłącznie badaną pętlę. Wczytanie pliku wykonaliśmy wcześniej,
// bo szybkość dysku nie jest przedmiotem tego benchmarku.
const auto start = std::chrono::steady_clock::now();
const std::int64_t checksum = calculate_checksum(values, rounds);
const auto stop = std::chrono::steady_clock::now();
```

To jest fragment, a nie kompletny program. Zwróćcie uwagę na komentarz: najpierw podaje cel decyzji, a dopiero potem wskazuje, co zostało umieszczone poza pomiarem.

4️⃣ Rozgrzewka

Pierwsze wykonanie może różnić się od następnych. Program i dane dopiero trafiają do odpowiednich obszarów pamięci, system przygotowuje potrzebne zasoby, a w niektórych środowiskach uruchamiane są dodatkowe

mechanizmy wykonawcze. Dlatego wykonujemy kilka prób rozgrzewkowych, których nie zapisujemy jako właściwych wyników.

Rozgrzewka nie jest sposobem na potajemne wyrzucenie niełądnego wyniku. Jej liczba musi być ustalona przed eksperymentem i opisana w raporcie.

> Pierwsza naleśnikowa prawda informatyki: pierwszy naleśnik często wychodzi dziwnie. Rzetelny tester nie wyrzuca go przez okno i nie twierdzi, że nigdy nie istniał — zapisuje w procedurze, że była to próba rozgrzewkowa.

5 Wiele właściwych prób

Pojedynczy czas może zostać zakłócony przez system operacyjny, antywirusa, aktualizację, komunikator albo proces, który właśnie przypomniał sobie o swoim istnieniu. Seria prób pokazuje, czy wynik jest typowy, czy przypadkowy.

W tej lekcji zachowujemy wszystkie surowe wyniki. Możemy podać minimum i medianę jako krótkie podsumowanie, ale dokładnej analizie średniej, mediany, rozrzutu i wartości odstających poświęcimy lekcję 09.

Nie wybieramy jednego najlepszego wyniku tylko dlatego, że najbardziej nam się podoba

6 Walidacja wyniku

Obie implementacje muszą zwrócić ten sam oczekiwany `checksum`. Jeżeli wynik jest błędny, czasu nie uznajemy za poprawny pomiar wydajności.

Kompilator C++ może usuwać obliczenia, których rezultat nie wpływa na zachowanie programu. Wypisanie i sprawdzenie sumy kontrolnej sprawia, że wynik jest obserwowalny. Nie daje to magicznej ochrony przed każdym rodzajem optymalizacji, ale w naszym eksperymencie pokazuje podstawową zasadę: **benchmarkowana praca musi mieć sprawdzalny skutek**.

> Program, który niczego nie liczy, z natury bywa niezwykle szybki.

7 Release w C++

Wersję C++ mierzymy w konfiguracji Release z włączoną optymalizacją. Debug służy do diagnozowania programu, a nie do ogłaszania jego maksymalnej wydajności. Porównywanie Pythonowego wykonania z C++ Debug przypomina ocenianie sprintera biegnącego w gumowcach — wynik jest mierzalny, tylko pytanie badawcze zrobiło się podejrzane.

Debug można uruchomić jako osobny eksperyment pokazujący wpływ konfiguracji. Nie wolno jednak po cichu mieszać go z tabelą Release.

8 Jednostki i format

Oba programy wypisują te same pola:

```
language;trial;time_ns;checksum;correct
```

Jednostka znajduje się w nazwie `time_ns`, a komórka zawiera samą liczbę. `ns` oznacza nanosekundy. Duża liczba cyfr po pomiarze nie gwarantuje jednak równie dużej dokładności całego eksperymentu.

Surowe rekordy zachowujemy bez ręcznego poprawiania. Dopiero zwalidowane dane łączymy w tabelę do analizy.

Pomiar w Pythonie

Python używa `time.perf_counter_ns()`. Jest to precyzyjny zegar przeznaczony do mierzenia upływu czasu, a jego wynik otrzymujemy w nanosekundach jako liczbę całkowitą.

```
# perf_counter_ns mierzy upływ czasu zegarem monotonicznym. Wczytanie pliku
# pozostaje poza pomiarem, ponieważ badamy pętlę obliczeniową, nie dysk.
start = time.perf_counter_ns()
checksum = calculate_checksum(values, rounds)
stop = time.perf_counter_ns()
```

To również jest fragment. Zegar monotoniczny nie cofa się wskutek korekty godziny systemowej. Nie mówi nam, która jest godzina; mówi, ile czasu upłynęło między `start` i `stop`.

W Pythonie możemy spotkać `timeit`. To przydatne narzędzie automatyzujące powtórzenia małych fragmentów kodu. W naszym ćwiczeniu używamy jednak jawnej pętli pomiarowej, ponieważ chcemy samodzielnie zobaczyć rozgrzewkę, walidację, numer próby i ustrukturyzowany output. `timeit` nie jest zakazane — po prostu dzisiaj nie chowamy mechanizmu pod maską.

Pomiar w C++

C++ używa poznanego już `std::chrono::steady_clock`. Czas przeliczamy na nanosekundy przez `duration_cast<std::chrono::nanoseconds>`.

Konfigurację Release i wersję kompilatora trzeba zapisać w metryczce środowiska. Sam napis „C++” jest mniej dokładny niż opis samochodu jako „metalowy”.

Co nadal może poruszać wynikami?

Nawet poprawny benchmark nie zamienia komputera w sterylne laboratorium. Na czasy wpływają między innymi:

- inne procesy i decyzje planisty systemu operacyjnego;
- zmiana częstotliwości procesora i tryby oszczędzania energii;
- temperatura i ograniczanie taktowania po rozgrzaniu;
- stan pamięci podręcznej procesora;
- rozmieszczenie danych w pamięci;
- optymalizacje kompilatora;
- interpreter i wersja Pythona;
- antywirus, indeksowanie i aktualizacje działające w tle.

Nie musicie dzisiaj diagnozować potoku procesora, predykcji skoków i każdego poziomu cache. Musicie natomiast wiedzieć, że wynik czasu nie jest stałą fizyczną programu. To obserwacja wykonana w opisanych warunkach.

> **Ciekawostka:** procesor może chwilowo przyspieszać taktowanie, a po wzroście temperatury je obniżać. Dlatego seria 200 prób uruchomiona natychmiast po starcie chłodnego komputera może opowiadać również historię jego układu chłodzenia. Wentylator nie jest recenzentem Waszego kodu, choć czasem brzmi bardzo krytycznie.

🧪 Studium przypadku — jedna praca, dwa silniki

Demonstracyjny zestaw znajduje się w katalogu `TiDPI/Eksperymenty/L07_Rzetelny_benchmark`. Zawiera generator wspólnego pliku, program C++ i program Python. Kod służy do uruchomienia na lekcji; Waszym zadaniem jest zbudowanie własnego eksperymentu, a nie skopiowanie demonstracji.

Przebieg demonstracji:

- generator tworzy jeden plik z danymi i zapisuje użyte `count` oraz `seed`;
- oba programy odczytują ten sam plik;
- oba obliczają tę samą sumę kontrolną;
- rozgrzewki wykonują pracę, ale nie trafiają do tabeli;
- właściwe próby mierzą tylko pętlę obliczeniową;
- każdy rekord zawiera język, numer próby, czas, checksum i informację o poprawności;
- błędny wynik przerywa przebieg kodem `1`, a błąd danych lub konfiguracji kodem `2`.

Jeżeli Python i C++ zwrócą różne sumy kontrolne, nie pytamy jeszcze „który jest szybszy?”. Najpierw pytamy „który wykonuje inne zadanie albo ma błąd?”.

🚧 Najczęstsze pułapki

- generowanie osobnych danych w każdym języku i nazywanie ich identycznymi;
- mierzenie w C++ konfiguracji Debug jako docelowej wydajności;
- włączenie odczytu pliku do jednego pomiaru, ale nie do drugiego;
- brak sprawdzenia wyniku;
- jedna próba i wniosek o całym języku;
- wyrzucanie niewygodnych wyników bez wcześniej ustalonej reguły;
- porównanie różnych algorytmów pod nagłówkiem „C++ kontra Python”;
- podanie średniej bez surowych wyników;
- zmiana liczby iteracji tylko w szybszym programie;
- uruchomienie benchmarków jednocześnie i pozwolenie im na walkę o procesor;
- pomylenie rozdzielczości zegara z dokładnością eksperymentu;
- napisanie „Python jest wolny”, gdy zmierzono jedną ręczną pętlę w jednej wersji interpretera.

> Jeśli wynik jest spektakularny, tester nie krzyczy od razu „Eureka!”. Najpierw sprawdza, czy przypadkiem nie mierzył pustej pętli. Dopiero potem wolno mu krzyczeć — najlepiej po sprawdzeniu jednostek.

🧰 Ćwiczenia i eksperyment

Wykonujcie poziomy kolejno. Każdy poziom wykorzystuje jeden wspólny plik danych i jedno sprawozdanie. Prostszym eksperymentem można wykazać poprawną realizację podstaw, a wyższe oceny wymagają nie większej liczby przypadkowych pomiarów, lecz lepszego projektu i mocniejszych dowodów.

🟢 Poziom A — buduję poprawny rdzeń

- Przygotujcie generator wspólnego pliku zawierającego co najmniej `10 000` liczb całkowitych.
- Zapiszcie w raporcie liczbę elementów, seed generatora i krótki odcisk albo checksum danych.
- Zaimplementujcie w C++ i Pythonie tę samą ręczną pętlę obliczeniową uzgodnioną z prowadzącym.
- Oba programy muszą odczytywać ten sam plik i zwracać identyczny wynik kontrolny.
- Oddzielcie wczytanie i przygotowanie danych od mierzonej operacji.
- Wykonajcie co najmniej dwie rozgrzewki i dziesięć właściwych prób.
- C++ uruchomcie w konfiguracji Release.
- Zapiszcie surowe rekordy obu programów w ustrukturyzowanej postaci.

📦 *Do oddania — poziom A*

Przekażcie poniższy zestaw plików. W miejscu Kryptonim wpiszcie Wasz przydzielony kryptonim TiDPI.

```
Kryptonim\
└── TiDPI_07\
    ├── raport.md
    ├── raport.docx albo raport.pdf
    ├── benchmark.cpp
    ├── benchmark.py
    ├── generate_data.py
    ├── dane\
    │   └── input.txt  (lub procedura odtworzenia w raporcie)
    ├── wyniki\
    │   ├── A_cpp_raw.txt
    │   └── A_python_raw.txt
    └── images\  (jeśli używacie obrazów)
```

Dołączcie źródła obu implementacji i generatora oraz wspólne dane albo jednoznaczną procedurę odtworzenia tego samego pliku. W raporcie zapiszcie count, seed, odcisk/checksum, środowisko, Release, granice pomiaru, co najmniej dwie rozgrzewki i dziesięć właściwych prób. Surowe rekordy obu implementacji muszą zawierać kontrolę zgodności wyniku. Oddajecie jedno sprawozdanie z oznaczeniem faktycznie wykonanego zakresu. Nazwy dodatkowych plików pokazane w drzewie są wzorcem organizacyjnym; jeżeli Wasz program używa innych nazw, zachowajcie działające odwołania i wskażcie te pliki w raporcie. Zapis <...> oznacza nazwę do zastąpienia, a nie dosłowną nazwę pliku. Wszystkie obrazy użyte w raporcie umieśćcie w images\ i odwołujcie się do nich względnie, np. ![Dowód D01](images/D01.png). Jeżeli nie używacie obrazów, katalog images\ nie jest wymagany. Pozostałe załączniki również linkujcie względnie.

🛡️ Poziom B — dowodzę rzetelności

- Zautomatyzujcie uruchomienie obu programów i zapis `stdout`, `stderr` oraz kodów wyjścia.
- Zweryfikujcie nagłówek, typy, liczbę prób, jednostki i zgodność checksumów.
- Wykonajcie kontrolowany przypadek błędny: zmieńcie jedną implementację albo dane tak, aby walidator odrzucił porównanie.
- Zachowajcie wszystkie surowe czasy i policzcie minimum oraz medianę osobno dla obu implementacji.
- Wyjaśnijcie, dlaczego minimum i mediana odpowiadają na nieco inne pytania.

- Powtórzcie cały eksperyment w osobnym przebiegu i sprawdźcie, czy główny wniosek pozostaje taki sam.

Ukończenie poziomów A i B pozwala uzyskać najwyżej ocenę 4.

📦 *Do oddania — poziom B*

Zachowajcie materiały z wcześniejszych wykonanych poziomów i uzupełnijcie ten sam raport. Poniżej pokazano pliki dodawane lub aktualizowane na tym poziomie.

```
Kryptonim\
└── TiDPI_07\
    ├── raport.md  (uzupełniony o A–B)
    ├── raport.docx albo raport.pdf  (aktualny eksport)
    ├── run_benchmark.py
    ├── validate_results.py  (jeśli walidator jest osobnym skryptem)
    └── przebiegi\
        ├── <przebieg_1>\
        ├── <przebieg_2>\
        └── <przypadek_bledny>\
```

Dla każdego przebiegu zachowajcie osobno surowe cpp_stdout.txt, cpp_stderr.txt, python_stdout.txt, python_stderr.txt i exit_codes.txt z kodami obu programów. Dołączcie wynik walidacji (np. walidacja.txt), a dla zaakceptowanych pomiarów także results.csv. Zachowajcie źródła lub zmienione dane przypadku błędnego, aby można było odtworzyć odrzucenie porównania. Walidator może być częścią automatu. Minimum, medianę, ich znaczenie i porównanie dwóch pełnych przebiegów zamieśćcie w raporcie.

🔧 Poziom C — badam wpływ jednej zmiennej

Wybierzcie dwa z poniższych rozszerzeń:

- porównajcie C++ Debug i Release, nie mieszając ich w jednej kategorii;
- wykonajcie co najmniej trzy rozmiary danych i sprawdźcie, jak zmienia się czas;
- porównajcie brak rozgrzewki z ustaloną rozgrzewką;
- zbadajcie ręczną pętlę i funkcję biblioteczną, opisując je jako inne implementacje;
- zmieńcie liczbę rund tak, aby pojedyncza próba trwała wystarczająco długo, i uzasadnijcie wybór.

W każdym rozszerzeniu zmieniajcie jedną zaplanowaną rzecz, a pozostałe warunki utrzymujcie możliwie stałe. Poprawne poziomy A–C otwierają drogę do oceny 5.

📦 *Do oddania — poziom C*

Zachowajcie materiały z wcześniejszych wykonanych poziomów i uzupełnijcie ten sam raport. Poniżej pokazano pliki dodawane lub aktualizowane na tym poziomie.

```
Kryptonim\
└── TiDPI_07\
    ├── raport.md  (uzupełniony o A–C)
    ├── raport.docx albo raport.pdf  (aktualny eksport)
    └── rozszerzenia\
        └── C\
            ├── <badanie_1>\  (źródła, parametry i przebiegi)
            └── <badanie_2>\  (źródła, parametry i przebiegi)
```

Dołączcie dane i komplet dowodów dla dwóch wybranych rozszerzeń, w układzie przebiegów jak w B. Oznaczcie konfiguracje Debug/Release, rozmiary, rozgrzewki, implementacje lub liczbę rund zgodnie z wyborem. Źródła wspólne można wskazać ścieżką względną; zmodyfikowane warianty zachowajcie osobno. W raporcie wskażcie zmienną badaną i warunki stałe.

🚀 Poziom D — projektuję mocniejszy benchmark

Ocena 6 wymaga wartościowego rozszerzenia ponad program, na przykład automatycznej kalibracji liczby iteracji, losowania kolejności uruchomień, zapisu pełnej metryczki środowiska, wykresu rozkładu czasów albo zbadania wpływu funkcji bibliotecznych implementowanych poza interpreterem Pythona. Rozszerzenie musi mieć hipotezę, kontrolowane zmienne, walidację wyniku i ograniczenia. Samo uruchomienie pętli tysiąc razy nie jest wiedzą ponadprogramową — jest bardzo wytrwałym naciskaniem przycisku.

🥖 *Do oddania — poziom D*

Zachowajcie materiały z wcześniejszych wykonanych poziomów i uzupełnijcie ten sam raport. Poniżej pokazano pliki dodawane lub aktualizowane na tym poziomie.

```
Kryptonim\
└── TiDPI_07\
    ├── raport.md  (uzupełniony o A–D)
    ├── raport.docx albo raport.pdf  (aktualny eksport)
    └── rozszerzenia\
        └── D\
            ├── <źródła i konfiguracja rozszerzenia>
            └── <surowe wyniki, walidacja i logi>
```

Zestaw dopasujcie do wybranego rozszerzenia: zachowajcie np. wyniki kalibracji, kolejność uruchomień albo metryczkę środowiska. Jeśli tworzycie wykres rozkładu, umieśćcie go w images\ i dołączcie dane, na których powstał. Hipotezę, kontrolowane zmienne, walidację i ograniczenia opiszcie w raporcie.

📝 Sprawozdanie

Oddajecie ostatnie indywidualne sprawozdanie z cyklu lekcji 01–07. Raport przygotujcie w Markdownie zgodnie z lekcją 01. Może zostać przekazany internetowemu agentowi AI do wstępnej analizy, dlatego musi być całkowicie zanonimizowany.

🔒 Anonimizacja

- użyjcie wyłącznie przydzielonego kryptonimu TiDPI;
- usuńcie imię, nazwisko, e-mail, nazwę konta i prywatne ścieżki z treści, metadanych, nazw plików, terminala i zrzutów;
- nie umieszczajcie w paczce prywatnej mapy kryptonimów;
- przed oddaniem otwórzcie raport i dowody tak, jak zobaczy je osoba z zewnątrz.

🧱 Wymagana struktura raportu

- kryptonim TiDPI i wykonany poziom;
- cel, pytanie badawcze i oczekiwany wspólny wynik;
- metryczka sprzętu, systemu, kompilatora, konfiguracji C++ i wersji Pythona;
- opis danych wejściowych oraz dowód, że były wspólne;
- opis obu implementacji i granic pomiaru;
- liczba rozgrzewek, prób i rund;
- walidacja wyników oraz przypadek błędny;
- surowe wyniki;
- minimum i mediana dla każdej implementacji;
- porównanie wyników bez nieuprawnionych uogólnień;

- źródła zakłóceń i ograniczenia;
- rejestr dowodów `D01`, `D02`...;
- wykonane rozszerzenia;
- wnioski;
- wykorzystanie AI i sposób sprawdzenia jego pomocy.

📦 Artefakty do oddania

- raport `.md` oraz wygenerowany `.docx` albo `.pdf`;
- źródła C++ i Python;
- generator i wspólny plik danych albo jednoznaczna procedura jego odtworzenia;
- skrypt automatyzujący przebieg;
- surowe wyniki obu implementacji;
- zwalidowany CSV;
- logi i kody zakończenia;
- dowody użyte w raporcie.

📊 Progi ocen

Maksymalna ocena	Wymagany zakres	Najważniejsze dowody
2	częściowy poziom A	uruchomienie obu programów i próba zapisu wyników
3	poprawny poziom A	wspólne dane, Release, rozgrzewka, 10 prób, zgodny checksum i surowe rekordy
4	poziomy A–B	automatyzacja, walidator, przypadek błędny, minimum, mediana i powtórzony przebieg
5	poziomy A–C	dwa kontrolowane rozszerzenia i uzasadnione wnioski
6	poziomy A–C oraz wartościowy poziom D	wiedza ponadprogramowa, mocniejszy projekt eksperymentu i obrona decyzji

Niezależnie od poziomu oceniane są: poprawność programów, rzetelność porównania, jakość dowodów, anonimizacja, zgodność raportu z plikami oraz Wasze rozumienie pracy. Agent AI może pomóc nauczycielowi odnaleźć dowody, ale nie wystawia samodzielnie oceny. Jeżeli nie potraficie wyjaśnić kodu, pomiaru albo wniosku, dokument nie dowodzi Waszych umiejętności.

🏁 Co powinniście zapamiętać?

- Benchmark jest kontrolowanym eksperymentem, a nie konkursem na najmniejszą liczbę.
- Porównujemy konkretne implementacje w opisanym środowisku, nie abstrakcyjne języki.
- Obie implementacje muszą dostać te same dane i wykonać to samo zadanie.
- Granice pomiaru wynikają z pytania badawczego.
- Rozgrzewka i wiele prób ograniczają wpływ przypadkowych warunków.
- Surowe wyniki zachowujemy; nie wybieramy ręcznie ulubionego pomiaru.
- Błędny wynik unieważnia pomiar wydajności.
- C++ benchmarkujemy w Release, a konfigurację zawsze zapisujemy.
- Wynik opisujemy wraz z warunkami i ograniczeniami.
- Rzetelny tester nie pyta tylko „kto wygrał?”, lecz przede wszystkim „czy wyścig był uczciwy?”.